Лабораторная работа
Параллельное умножение матриц с использованием OpenMP

Студент группы ИА332 Рубцов Роман Кириллович

Новосибирск, 2026

1 Цель работы

Реализация и сравнительный анализ эффективности двух алгоритмов параллельного перемножения матриц размера 1000×1000: ленточного и блочного. Исследование зависимости времени выполнения от количества потоков и стратегии работы с кэш-памятью.

2 Конфигурация стенда

- **Процессор:** Intel Core i5-10400F (6 ядер, 12 потоков, 2.90 ГГц).
- **Кэш:** L1 384 КБ, L2 1.5 МБ, L3 12 МБ.
- **ОЗУ:** 16 ГБ DDR4.
- **Среда:** GCC (флаги `-O3 -fopenmp`).

3 Алгоритмы

3.1 1. Ленточный алгоритм

Используется оптимизированный порядок циклов $i \to k \to j$.

- Во внешнем цикле фиксируется строка первой матрицы.
- Во внутреннем цикле j происходит проход по строке второй матрицы.
- **Преимущество:** Последовательный доступ к памяти (Spatial Locality), эффективная работа аппаратного префетчера (Hardware Prefetching) и векторизации (SIMD).
- **Параллелизация:** Директива `#pragma omp parallel for schedule(static, len_k)` раздает диапазоны строк разным потокам.

3.2 2. Блочный алгоритм

Матрица разбивается на подматрицы (блоки) размера 32×32.

- Размер блока подобран под размер L1 кэша ($32 \times 32 \times 8$ байт ≈ 8 КБ).
- Вычисления производятся внутри блоков, что позволяет многократно переиспользовать данные, загруженные в быстрый кэш, минимизируя обращения к медленной оперативной памяти.
- **Параллелизация:** Распараллеливание внешнего цикла по блокам строк `#pragma omp parallel for schedule(static)`.

4 Результаты измерений

Размер матриц: 1000×1000. Количество прогонов для усреднения: 10.

Таблица 1: Сравнение времени выполнения (в микросекундах)

Потоки	**Time_lent (Linear)**	**Time_blocks (Tiling)**
1	306 457	287 665
2	154 279	147 149
4	77 822	74 643
6	53 030	56 108
8	68 018	71 419
12	49 746	54 471
16	51 324	55 031
32	47 503	49 569

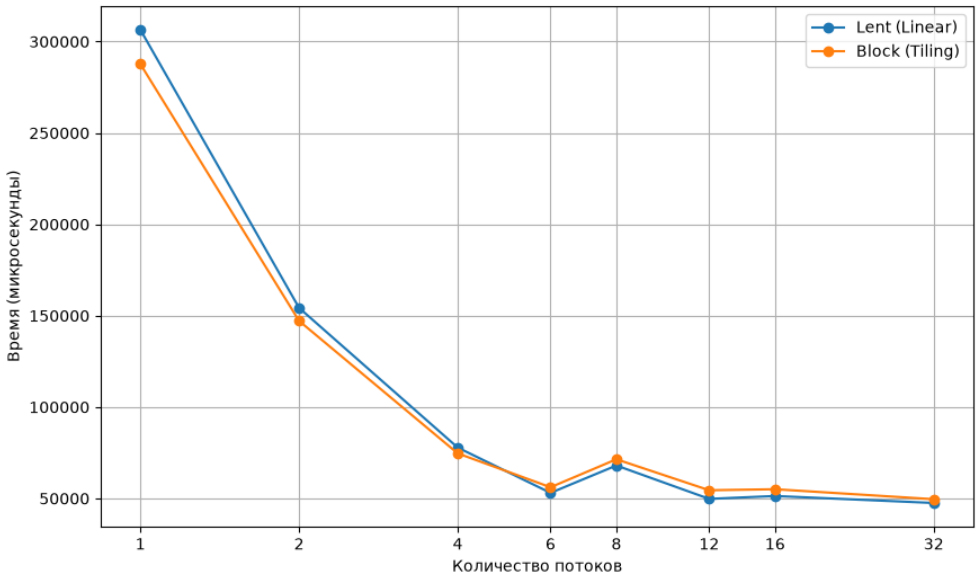


Рис. 1: График зависимости времени от числа потоков

5 Анализ данных

5.1 Сравнение однопоточной производительности

На 1 потоке **блочный метод оказался быстрее** (287 мс против 306 мс). Это подтверждает теорию: разбиение на блоки 32×32 позволяет данным эффективно удерживаться в L1 кэше, тогда как ленточный метод при размере строки 1000 элементов может вызывать больше промахов кэша.

5.2 Масштабируемость (Scaling)

- **1-6 потоков:** Оба алгоритма демонстрируют почти линейное ускорение. Время сокращается пропорционально количеству физических ядер (с 300 мс до 53 мс).
- **6-12 потоков:** Включается Hyper-Threading. Прирост производительности замедляется, но минимум достигается в районе 12 потоков (49 мс).
- **>12 потоков:** Дальнейшее увеличение числа потоков (16, 32) не дает прироста (время стагнирует около 50 мс).

5.3 Почему методы сравнялись?

Несмотря на теоретическое преимущество блочного метода, на большом количестве потоков ленточный метод (Lent) показал себя чуть лучше (47 мс против 49 мс на 32 потоках). **Причины:**

1. Размер матрицы 1000×1000 (8 МБ данных) полностью помещается в L3 кэш процессора (12 МБ). Это снижает штраф ленточный метода за работу с памятью.

2. Блочный метод имеет накладные расходы на организацию 6 вложенных циклов и вычисление границ `std::min`.

3. Ленточный метод с порядком $i - k - j$ идеально векторизуется компилятором в AVX-инструкции.

6 Вывод

Для матриц размера, помещающегося в L3 кэш процессора, оптимизированный ленточный алгоритм не уступает, а иногда и превосходит блочный метод по скорости распараллеливания. Блочный метод показывает преимущество в однопоточном режиме за счет лучшей работы с L1 кэшем. Предел производительности системы достигается на 12 потоках, после чего узким местом становится пропускная способность памяти.